SALT'S ANTI ALIASING / MINECRAFT FABRIC 26.3 / 25 OCTOBER 2026

DLSS implementation audit

Super Resolution, DLAA, motion data and rendering correctness

Main conclusion

The core camera reprojection and sampling path is coherent in the tested vanilla Vulkan scenes. The integration is still incomplete for all rendered content: particle velocity is missing, and the color-rejection mask used as a fallback is not a supported mitigation for the current K/M/L models.

This is a code-and-rendering audit of the current mod, including the newly added model selection and DLAA. It compares the actual Java, GLSL and JNI path with NVIDIA's current documentation, SDK headers and implementation. It includes real game outputs, raw GPU inputs, numerical reprojection checks and enlarged image crops.

Most important result	Assessment
Camera motion, units and signs	Passed the sampled numerical checks; 95th-percentile error below 0.003 input pixels.
Moving entities / animated geometry	Actual geometry velocity is present. Coverage is conditional on the captured renderer and stable vertex correspondence.
Particles and layered effects	Confirmed missing velocity in the tested flame region; not covered by the entity path.
Current-color rejection	High-priority design mismatch. NVIDIA advises against this input for the newer models.
Depth metadata / constants	One SDK warning confirmed; one native depth-subresource contract risk needs validation-layer testing.

Recommended order: correct unsupported mask usage and particle handling first; then harden native resource metadata and coverage diagnostics. Preserve the camera/jitter path that already passes. No production rendering code or normal user settings were changed during this audit.

HOW THE EVIDENCE WAS OBTAINED

1. Scope, build identity and method

Item	Audited setup
Game and platform	Minecraft 26.3; Fabric Loader 0.19.5; Java 25; Windows x64; Vulkan.
GPU and driver	NVIDIA GeForce RTX 3060 Ti, 8 GiB; NVIDIA 591.86.
DLSS stack	User-imported nvngx_dlss.dll 310.9.1.0; Streamline 2.14.1; mod-owned bridge ABI 3.
Public artifact	salts-anti-aliasing-mc26.3-1.1.jar. SHA-256 begins e1031416ded18d9f; full hashes are in section 21 and the evidence manifest.
Live session	10:20:32 to 10:30:14 UTC on 25 October 2026. Isolated game directory and a disposable diagnostic helper.
Capture coverage	797 output screenshots and 430 raw GPU buffer copies, including excluded exploratory runs. Accepted cases are identified by label.
Outputs tested	1280 x 720 and 1920 x 1080. Quality, Performance, Ultra Performance and DLAA; K/M/L; resizing, camera movement, geometry, particles, textures, water and hand composition.

Evidence levels

Measured means a value came from captured GPU buffers or frame data. **Observed** means a visual result was inspected. **Code-confirmed** means the implementation contains or lacks a specific path. **Risk** identifies a plausible integration problem without claiming that it caused a visible defect in these tests.

The public mod JAR was loaded by a normal Fabric client. The audit helper lived in a separate JAR, recorded inputs and scripted scenes; it did not replace the renderer. Jitter signs remained +1/+1 in the helper and phase override stayed zero. The native bridge therefore received the production jitter convention.

An early pig fixture was accidentally removed by its own setup command. Its four entity runs were discarded and replaced by entity-valid-* captures. A helper argument error also stopped the first suite before its hand test; native-hand-valid is the completed replacement. These failures are recorded rather than counted as rendering defects.

FROM SCENE RASTERIZATION TO FINAL COMPOSITION

2. The actual rendering path

Scene setup	SDK size + model; internal color / depth
World rasterization	Jittered projection; unjittered matrices retained
Motion generation	Depth reprojection + matched entity geometry
DLSS evaluation	Color + depth + velocity + constants + legacy mask
Native output	Full-resolution world; post effects / sharpening
Foreground composition	Hand MSAA and HUD after world reconstruction

GameRendererMixin starts the internal-resolution path before world rendering and updates the resolution-dependent global uniforms. The final world projection is jittered after vanilla's camera effects. VulkanSceneDlssController redirects the main scene target while the world is drawn, then generates velocity and its current-color rejection texture.

The controller records DLSS into a separate transient Vulkan command buffer, submits it, and blits its native-resolution output into the main target. The normal world output is therefore reconstructed before the first-person hand and HUD are composed. This is a deliberate architecture choice: the hand does not need to be reconstructed by DLSS.

Why scene ordering matters

A source color pixel, its depth, its velocity and the reported jitter must describe the same rasterized frame. A color feature rendered into this source without corresponding object motion is the primary remaining weakness. A later foreground layer avoids that requirement, but needs its own native-resolution anti aliasing and correct depth/composition behavior.

Evidence / references: C1 GameRendererMixin; C2 RenderRuntime; C3 VulkanSceneDlssController. NVIDIA Streamline integration [S2], manual integration [S6].

CORE IMAGE AND MOTION RESOURCES

3. Required inputs: contract comparison

Data	Current implementation	Assessment
Input color	Render-resolution RGBA8_UNORM, tagged ScalingInputColor; world scene before hand/HUD.	Present. SDR encoding assumptions need explicit compatibility limits.
Output color	Native-resolution RGBA8_UNORM storage-capable target, tagged ScalingOutputColor.	Present; 720p and 1080p output verified.
Depth	Render-resolution D32_FLOAT from the jittered world pass; reverse-Z declared.	Present. Native subresource aspect metadata is a separate concern.
Dense camera velocity	RG16_FLOAT; reconstruct from current jittered depth into unjittered current/previous transforms.	Present; sampled yaw/pitch/strafe errors below 0.003 px at p95.
Dynamic-object velocity	Overlay matched transformed entity vertices after camera flow; depth agreement test protects occlusion/alpha holes.	Present for intercepted geometry. Not universal.
Particle velocity	No particle history/velocity capture path found. Flame ROI stays effectively zero with a stationary camera.	Confirmed coverage gap.
Jitter	Halton 2/3, render-pixel offsets; JNI sends raw X and negated raw Y.	Present; tested sequence and output are consistent.
Frame identity / viewport	Fresh SDK token per evaluation; all tags/constants/evaluate use viewport 0.	Present; single-view integration.
Discontinuity reset	Temporal flags plus color-history invalidation reach the native reset flag.	Present for tested changes; coverage of other transitions remains to be tested.

NVIDIA's core SR contract is color, depth, motion and synchronized frame constants. A renderer can submit every required texture and still have incorrect reconstruction if parts of the visible color image lack the correct velocity. Presence of an RG16F texture is not proof of complete motion coverage.

Evidence / references: [S2] required resource tags and per-frame calls; [S3] constants; [S1] sections 3.6-3.8. Implementation C3-C8.

AVOIDING FALSE REQUIREMENTS

4. Other data: required, optional or unrelated?

Item	Current behavior	Interpretation
Motion scale / flags	1 / input width and height; camera motion included; 2D vectors. Jittered/dilated flags remain false.	Consistent with raw, render-resolution, unjittered pixel velocity.
Camera transforms	Unjittered projection and inverses; current-to-previous clip transform computed in JNI.	Present. Rebased coordinates are intentional, not missing player motion.
Camera position / basis / projection	Rebased origin, basis vectors, near/far, FOV and aspect sent. Pinhole offset not explicitly initialized.	Mostly supplied. SDK logs one invalid-value warning.
Exposure	useAutoExposure=true; no exposure texture; preExposure and exposureScale use SDK defaults of 1.	Accepted auto-exposure path; not a missing mandatory texture.
Color precision / encoding	8-bit UNORM and HDR=false.	Suitable only when source is perceptually encoded SDR. Linear/HDR shader packs need a different contract.
Current-color bias mask	R16_FLOAT mask generated from color-history differences and tagged on every DLSS evaluation.	Obsolete mitigation for the current models; should not be relied upon.
Normals / roughness / hit distances	Not generated for DLSS SR.	Not required core SR inputs. Their absence is not an SR defect.
Optical flow / frame-generation timing	No DLSS FG/MFG path included in this audit.	Separate features with additional requirements; not missing SR data.
Alpha upscaling	Left disabled; final game image is RGB with a separately composed foreground.	Reasonable. Enabling it would not supply missing translucent object motion.

Evidence / references: [S3] common constants; [S4] DLSSOptions; [S1] exposure, color and mask sections. C3, C9.

MEASURED: CORRECT IN SAMPLED SCENES

5. Camera and player movement vectors

The renderer reconstructs a camera-relative point using the inverse jittered projection/view transform and the actual D32 depth. It projects that point into unjittered current and previous frames. The velocity is the previous screen position minus the current screen position, expressed in input pixels. This removes sampling jitter from physical motion.

The convention used by the mod

motionPixels = (previousUV - currentUV) * inputSize
Streamline scale = (1 / inputWidth, 1 / inputHeight)
Camera translation is already included; the SDK is told not to add it again.

Motion	Samples	Median error px	p95 error px	Median flow px
Yaw	60,478	0.000829	0.002798	2.522
Pitch	61,013	0.000457	0.000997	1.473
Strafe	59,975	0.000251	0.001296	0.844

An independent NumPy implementation applied the captured transforms to sampled depth pixels and compared the expected result with RG16F GPU readback. The small residuals are consistent with half-float storage and floating-point calculation. This is strong evidence against a global sign, scale or jitter-subtraction mistake in the sampled camera path.

Player movement is camera movement in first person

Walking, strafing and camera rotation move the view even when the player body is hidden. The camera matrices already account for that. A separate player-position vector should not be added to these pixels. Third-person body animation is a different requirement and belongs in object velocity, discussed next.

Limits: this validates the motion shader against captured transforms, not every possible game camera effect. It does not certify hurt/bob/nausea, unusual projections, VR or all camera transitions. Clear-depth sky uses rotation without translation parallax; animated clouds remain a separate shading case.

Evidence / references: Camera-yaw/pitch/strafe raw GPU buffers and input logs; analysis.json. C4-C5; [S1] 3.6; [S3].

MEASURED COVERAGE, WITH CONDITIONS

6. Dynamic entities: actual geometry velocity



Different motion phases are shown; these are not pixel-identical A/B poses. Bright pixels indicate motion magnitude; black means near-zero movement, not missing objects. Raw GPU rows are vertically reversed for display to match Minecraft screenshots; vector values are unchanged.

Accepted test	Matched triangle vertices	Nonzero flow pixels / frame
entity-valid-vectors	252	1,316 to 1,669
entity-valid-disabled	0	0 above 0.001 px
entity-valid-restored	252	1,260 to 1,549

EntityMotionGeometry stores emitted transformed positions and matches them to the previous frame by stable entity/block identity, model/layer and occurrence. This can represent body translation, model rotation and skeletal movement rather than only an entity-center displacement.

The matched geometry is rasterized over the dense camera field. The fragment shader only replaces velocity where its depth agrees with the visible scene depth. Occluded geometry and alpha holes retain the camera field. This is materially better than assigning a constant velocity to an entity bounding box.

Evidence / references: Corrected entity-valid-* captures only. C6 EntityMotionGeometry; C7 VulkanEntityMotionRenderer; C8 entity_motion shaders.

CODE-CONFIRMED LIMITATIONS, PARTLY EXERCISED

7. Animated geometry and coverage boundaries



Chest lid animation and third-person arm swing were exercised in the live client. Enlarged crops use nearest-neighbor scaling; they preserve the captured pixels.

The chest-motion run emitted 2,424 matched triangle vertices per captured frame in the surrounding scene. Third-person-swing emitted 1,308-1,560. Those totals include other captured scene geometry, so they demonstrate an active path rather than a per-object correctness percentage. The isolated moving-pig toggle gives the stronger controlled evidence.

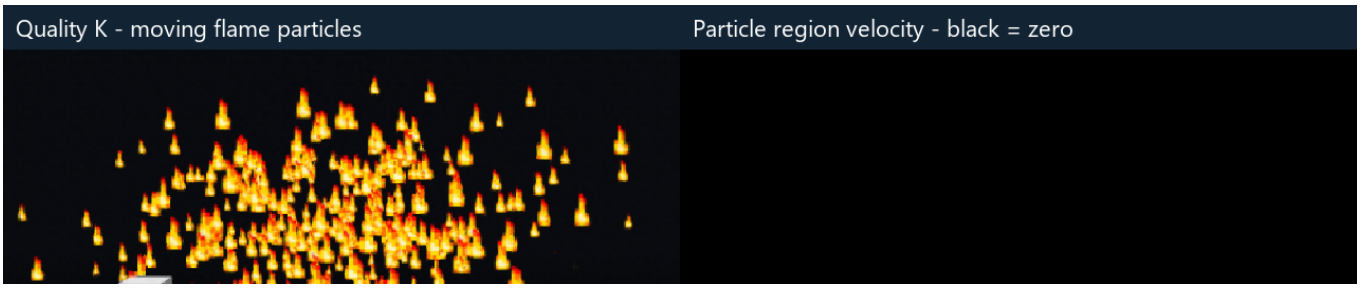
Path / condition	Coverage and consequence
Vanilla model submissions	Intercepted at Model.renderToBuffer. Includes normal entity/player model transforms.
Items, moving blocks, block entities	Additional submission hooks exist. Animated chest was exercised; every item/falling-block variant was not.
Particles and custom feature renderers	No general motion capture for all visible features. Text, beams, ropes, custom shaders and modded renderers need explicit coverage tests.
First appearance / topology change	Unmatched keys, changed position count, or non-quad count skip object velocity. Camera flow remains underneath.
262,144 captured-vertex limit	Later geometry stops being captured. No visible saturation warning or per-object coverage indicator exists.
Stable ordering assumption	Equal vertex count is necessary but not sufficient to prove semantic correspondence. Renderer/layer reordering needs regression coverage.

Recommendation: expose diagnostic counters for captured, matched, skipped and capped geometry. Prioritize stable previous transforms or a native motion pass for known renderer families. Add a stress scene that deliberately exceeds the cap before choosing a new limit.

Evidence / references: C6 capture/captureSubmitted/forEachMatched and MAX_VERTICES; C10 submission mixins. Coverage gaps are not proof that every listed feature visibly fails.

P1 / MISSING RENDERING DATA

8. Particles: the strongest confirmed gap



Flame particles visibly occupy the color image. The corresponding normalized particle ROI has no meaningful velocity. Raw GPU rows are vertically reversed for display to match the screenshot; both panels show the same dispatch region.

The stationary-camera flame test records moving sprites in color while the dense motion pass sees the backdrop depth. Particle geometry does not enter the matched entity path. In all 24 captured frames, the flame ROI had zero pixels above 0.001 input pixels of motion; its maximum magnitude was approximately 0.0000066 pixels, numerical residue.

This is a confirmed input mismatch. It creates conditions in which temporal reconstruction must infer sprite motion or discard history. Fine sparks may become unstable, soften, lose detail or trail. The captured images show model-dependent particle appearance; this audit does not assign a numerical ghosting score because the particle sequences were not deterministically replayed across models.

Preferred solution

Track stable particle identity plus previous position, size and billboard orientation. Generate current-to-previous motion at the same jittered sample locations as particle color. Establish an explicit depth/alpha policy so transparent particle velocity does not blindly replace the motion of opaque geometry underneath it.

Practical alternative

Render selected problematic particle classes after DLSS at native output resolution. This costs more particle rasterization and requires correct depth/occlusion, blending, fog and hand ordering. It is often a simpler way to avoid feeding unmatched moving sprites into the temporal source. Do not merely increase a global reactive slider.

Evidence / references: particle-k raw motion and output; C4, C6-C8; [S1] 3.6 and 3.15. No particle velocity producer was found in the production mixin list.

P1 / UNSUPPORTED MITIGATION AND MISLEADING CONTROLS

9. The legacy rejection-mask mismatch



The mod really generates a nonzero R16F current-color mask. Raw GPU rows are vertically reversed for display to match the screenshot. A nonzero texture does not demonstrate that K/M/L apply it.

VulkanTemporalColorRejectionRenderer compares the previous color with a current 3x3 envelope, reprojected by motion plus jitter difference. The resulting continuous mask is tagged as BiasCurrentColorHint by the bridge. Advanced DLSS controls expose its strength, threshold and maximum.

The current NVIDIA guide, section 3.15, explicitly advises that this buffer should not be used with the latest models and says only the older F preset supports it. Streamline still forwards the tag to NGX, but that API plumbing is not evidence of modern-model support. K, M and L should therefore not rely on this path as a fallback for missing velocity.

Correction to the earlier mitigation assumption

A color-history rejection mask is not a substitute for complete motion data in this integration. The report does not claim a deterministic on/off experiment proved that every driver ignores the mask; the current documented contract is enough to stop treating it as a supported fix.

Recommended change: remove this input and its image-quality controls for modern presets unless NVIDIA supplies supported guidance. Keep a diagnostic visualization only if useful. Re-test particles and moving assets with correct velocity or native-resolution composition. Do not replace it with another undocumented mask based only on an exposed SDK tag name.

The previous-color and R16F mask targets cost about 2.34 MiB at 720p Quality, 5.27 MiB at 720p DLAA, and 11.87 MiB at 1080p DLAA, plus copies and a full-screen pass. These are removable costs if the modern path no longer consumes them.

Evidence / references: [S1] 3.15, printed p30 / PDF p40; [S5] forwards the hint. C3:364-383, C9, C11 NumericTuningSetting:83.

MEASURED: NO BROAD SCENE WOBBLE IN THIS FIXTURE

10. Jitter, sampling and static stability

Mode at 720p output	Input size	Phases	Mean edge temporal SD
Quality	853 x 480	19	0.775
DLAA	1280 x 720	8	0.245
Performance	640 x 360	32	1.552
Ultra Performance	427 x 240	72	1.279



Static white blocks against a dark backdrop. The crops are enlarged without smoothing; DLAA has more source samples and cleaner edge stability.

Each mode used 32 output frames. The edge mask was derived from the mean image gradient within x=445..834, y=315..409; SD is measured in 8-bit color code values. Mean whole-crop SD was 0.070 in Quality and 0.025 in DLAA. Edge-only averages above avoid hiding edge variation in the large flat backdrop.

Quality uses 19 phases at 853 x 480 because the implementation rounds the area-ratio formula upward; exact 1.5x input scaling at 1080p output gives 18. DLAA uses eight. These follow the intended sample-coverage logic. The Ultra Performance sample covers only 32 of its 72 phases, so this is not a full-cycle ranking of Performance against Ultra Performance.

The CPU jitter values are centered Halton offsets. The projection applies positive X and negative Y clip displacement, and JNI reports the matching image-space offset. The velocity field remains unjittered. No evidence here supports reversing Y globally again.

Evidence / references: static-* input logs and screenshots; analysis.json; C4-C5 and DlssOptimalSettings. [S1] 3.7.

CORRECT SEPARATION; UNRESOLVED LAYERED MOTION

11. Depth, transparency, hand and HUD



Underwater camera sweep and first-person foreground composition were inspected in game. The hand is a later native-resolution layer, not part of world DLSS history.

The world source uses D32_FLOAT reverse-Z depth; the bridge declares inverted depth. Motion samples the same jittered depth field with nearest filtering. The per-entity overlay tests visibility against it, which avoids writing velocity through many occluders and cutout holes.

A single depth/velocity pair cannot fully describe several alpha-blended layers moving independently through the same pixel. The current implementation mainly has the opaque depth plus the composited scene color. Water animation, translucent sprites and mixed foreground/background motion need targeted handling; changing a global vector scale cannot solve that representation problem.

Foreground path is intentional

The hand and HUD are drawn after reconstruction. native-hand-valid allocated 29,491,200 bytes of hand AA targets at 1280 x 720 with 4x MSAA. This confirms that the separate native hand path was active. Its lack of hand motion vectors in the DLSS world input is not a defect, because its color is not in that input either.

Recommended validation

Add overlapping moving glass, alpha-cutout leaves, dropped items, nameplates and additive particles against a high-contrast background. Verify color/depth/velocity coverage per layer. If a layer is moved after upscaling, ensure native-size depth, fog and blending are reproduced; do not resolve one artifact by drawing through walls.

Evidence / references: C1-C3; C7-C8; C12 VulkanHandAaController. Underwater observations are visual coverage, not a quantitative transparency-quality certification.

NO MISSING EXPOSURE TEXTURE BY ITSELF

12. Exposure and color encoding

The bridge explicitly enables auto exposure and sets colorBuffersHDR=false. It does not tag an exposure texture. Streamline documents this as a valid auto-exposure route. The options retain preExposure=1 and exposureScale=1. There is no evidence that a separate pre-exposure compensation is required by the tested vanilla SDR renderer.

Case	What should happen	Current assessment
Vanilla perceptually encoded SDR	Bounded source color with matching SDR processing.	RGBA8_UNORM plus HDR=false is consistent with this contract. No gross color shift was observed.
Linear-light source or HDR shader pack	Preserve linear/HDR range and precision; declare HDR processing.	Not implemented as a distinct DLSS color path and not validated here.
External pre-exposure	Report the same factor used to scale scene luminance.	No such engine factor was identified in this path; future HDR work must trace it explicitly.
K versus newer model exposure	Respect model-specific support; avoid assuming all manual exposure controls work everywhere.	The guide limits supplied exposure support to J/K and describes automatic exposure behavior for newer models.

NVIDIA prefers early post-processing placement and warns that linear color sent through the SDR path can introduce banding or color shifts. This does not mean an SDR vanilla integration must blindly enable HDR: the actual source encoding determines the correct flag. An RGBA8_UNORM format name alone does not prove whether its values are linear light or perceptually encoded.

Recommended guardrail

Document the supported vanilla SDR path. For shader-pack support, test a known luminance ramp and saturated patches before/after DLSS, including exposure changes and dark scenes. Add a high-precision target and correct exposure/pre-exposure handling only when the upstream renderer actually supplies that contract.

Evidence / references: [S1] 3.1.2, 3.9-3.10; [S2] auto-exposure note; [S4] option defaults. C3 and native bridge:281-282.

P3 CONFIRMED CLEANUP; BROADER CAMERA TESTS STILL NEEDED

13. Camera constants and their warning

The native bridge fills unjittered projection transforms, current/previous clip transforms, near/far, FOV, aspect and camera basis. cameraPos=(0,0,0) is intentional: the world matrices are rebased around the current camera, and inter-frame translation is subtracted in double precision before a small float offset is uploaded. This avoids losing local precision far from the origin.

Confirmed SDK warning

The Streamline log reports that consts.cameraPinholeOffset should not be left invalid. The bridge never assigns it, and its default float2 contains INVALID_FLOAT values. Initialize a centered camera offset explicitly to {0,0}.

The SDK header labels the pinhole offset optional, but its common validation still warns when the value is invalid. This is a real initialization omission with a straightforward fix. It is not proven to be the source of a visible artifact, especially because the SR path receives complete camera motion.

What remains uncertain

The near plane is fixed at 0.05. FOV is inferred from vanilla's camera projection while the final matrix may include hurt, bob or nausea transforms. Those choices are coherent in ordinary views, but changing projection conventions requires matching metadata. The audit did not certify every effect, camera mod, mount, portal, or orthographic view.

Acceptance tests

Render a fixed geometric calibration scene through view bob, hurt, sprint-FOV changes, nausea and perspective switches. Compare depth-derived flow against independent projected landmarks. Confirm no INVALID_FLOAT values, finite inverses, valid aspect/FOV, and a reset at a genuine discontinuity. Do not reset every frame during continuous FOV animation merely to hide an error.

Evidence / references: C4-C5; native bridge:432-458; Streamline log warning at 03:20:39 local time. [S3] constants; SDK common validation [S8].

MEASURED TRANSITIONS PASSED WITHIN SCOPE

14. Resets, resizing and native lifetime

Trigger	Current mechanism / evidence
Quality / DLAA / input size	SDK settings are queried; target sizes update; jitter sequence and history invalidate. 720p and 1080p transitions rendered successfully.
K/M/L model selection	Model is part of the settings cache and controller configuration. First evaluation invalidates color history; SDK recreates its feature when presets change.
Large camera discontinuity	Position or rotation thresholds call history invalidation. The explicit camera-cut test produced the rotation-cut path.
Entity-motion or texture tuning	Temporal input tuning can request a reset. Test toggle OFF/ON restored the geometry velocity path.
Leaving temporal mode	Temporal state resets; native feature retirement uses an inactive-frame window and a GPU fence.
Application shutdown	Metrics were saved; native integrations are shut down after the surface closes.

The helper recorded 38 reset evaluations during the session: 17 jitter-sequence changes, seven rotation cuts, seven tuning changes, four view initializations and three projection changes. The reason string is retained between events, so only records whose resetHistory flag is true were counted.

The SDK frame token is obtained afresh and remains monotonic even if Minecraft's local temporal frame counter resets. Tags, constants and evaluation use the same token and viewport. This avoids mixing resources from different frames at the native boundary.

Remaining validation gaps

Dimension transitions, disconnect/reconnect, resource reload, minimized/resumed presentation, camera-mod changes and device loss were not exercised in this audit. A short teleport or content discontinuity below a distance threshold may need an explicit engine event rather than a heuristic. Add those tests before claiming complete lifecycle coverage.

Evidence / references: C3, C4, C13 NativeFeatureRetirement; [S2], [S5] preset-change handling and delayed resource release.

P2 CODE-CONFIRMED CONTRACT RISK; ARTIFACT NOT PROVEN

15. Native Vulkan resource metadata

The bridge supplies image handles, actual Vulkan formats, usages, dimensions, views and GENERAL layout. The controller records broad synchronization2 barriers before and after a dedicated DLSS command buffer. Streamline manages its internal transitions, and the separate buffer avoids inheriting an unknown graphics pipeline state after evaluation.

Depth subresource chain is missing

image_resource() does not attach sl::SubresourceRange to the tagged resources. In the pinned Streamline Vulkan adapter, an absent range falls back to a COLOR aspect in its NGX resource descriptor. The actual depth image is D32_FLOAT and needs depth-aspect metadata.

The public SDK source makes this fallback visible. The existing VkImageView itself may still be a valid depth view; the problem is inconsistent metadata passed alongside it. The tested driver rendered without a native evaluation failure. No Vulkan validation layer was enabled, so this report classifies it as a native contract risk, not a demonstrated explanation for floating lines.

Recommended fix

Chain a persistent sl::SubresourceRange to each local sl::Resource for the duration of tagging/evaluation. Use DEPTH for D32 and COLOR for color/motion/mask, with mip 0, count 1, layer 0, count 1. Keep the chained structures alive through the call; do not return a pointer to a stack-local range from image_resource().

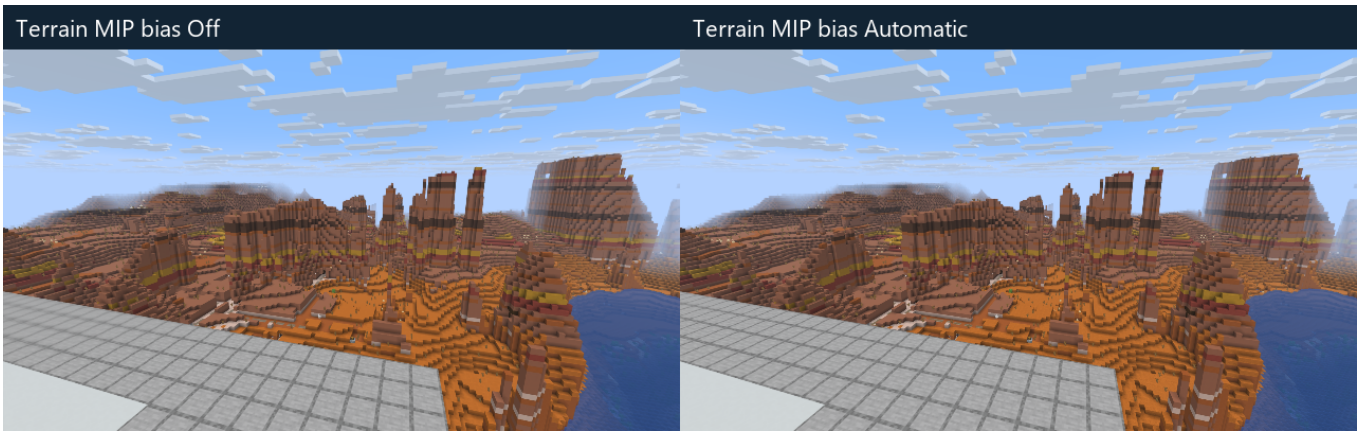
Recommended validation

Run a development build with Vulkan validation and synchronization validation. Exercise resizing, toggling DLAA, model changes, alt-tab and shutdown. Verify actual image layouts and ownership rather than assuming GENERAL forever. Only optimize the current ALL_COMMANDS / memory-read-write barriers after correctness is confirmed.

Evidence / references: Native bridge:84-103 and 413-425; C3:323-343. Streamline sl.dlss Vulkan cached resource fallback [S5]; Khronos aspect rules [S7].

P2 QUALITY TUNING, NOT A GLOBAL MOTION BUG

16. Terrain detail and sharpening



The same terrain viewpoint with optional MIP bias Off and Automatic. The mode is independent from the DLSS model selection.

The default tuning leaves MIP bias Off. Automatic can follow the render/output ratio with a temporal-upscaler adjustment and a user offset, rounded to quarter levels. The shader patch is restricted to Minecraft's core terrain fragment pipeline. Entity textures and arbitrary replacement shader paths do not receive the same treatment.

This is a scope limitation, not proof that the default is wrong. Minecraft's pixel textures and thin repeated patterns can become noisier with aggressive negative bias. Conversely, using native LOD assumptions at lower input resolution can discard texture detail before DLSS sees it. The right setting is asset-dependent.

Recommended change

Create a reference set containing distant leaves, rails, fence grids, signs, ore textures and texture-pack variants. Compare native DLAA with Quality at fixed camera and slow movement. Evaluate detail recovery and temporal variance together. Extend MIP handling only to the shader families that are known to support it.

Sharpening occurs after reconstruction through the mod's selectable sharpening path. The core audit used zero sharpening to avoid confusing reconstruction instability with halos. A supplementary 50% sample was captured; it is not used to diagnose native DLSS errors. Keep the user slider and compare at matched sharpness when assessing models.

Evidence / references: C14 VulkanTextureDetail; TextureDetailShaderPatch; NumericTuningSetting. [S1] 3.5 and 3.11.

VERIFIED REQUESTS; AVOID OVERSTATING ACTIVE-MODEL TELEMETRY

17. K/M/L, Auto and runtime identity

Control	What the implementation does
Quality slider	Stable native IDs select Quality, Balanced, Performance, Ultra Performance, Auto policy or DLAA. DLAA maps to eDLAA.
Model Auto	All mode-specific preset fields use eDefault; the imported NVIDIA runtime chooses its defaults.
Model K / M / L	The requested preset is assigned to DLAA, Quality, Balanced, Performance, Ultra Performance and Ultra Quality fields.
DLAA sizing	The Java bridge rejects a DLAA settings result that is not exactly output size. Native 720p and 1080p were observed.
Runtime files	The same imported runtime supports SR and DLAA. The public artifact contains only the mod-owned DLSS bridge, AMD runtime and FSR bridge.

The Streamline/NGX log records app-hint K, M and L requests during the relevant tests, and feature recreation occurs when presets change. This is better evidence than relying on the menu text alone. The SDK state exposed by this wrapper does not provide a verified active-model readback, and NVIDIA App overrides can supersede requests.

The expected current defaults are K for DLAA/Quality/Balanced, M for Performance and L for Ultra Performance. These are runtime defaults, not a promise that the mod controls driver overrides. The independent quality Auto option selects a scaling mode from output height; it is different from model Auto.

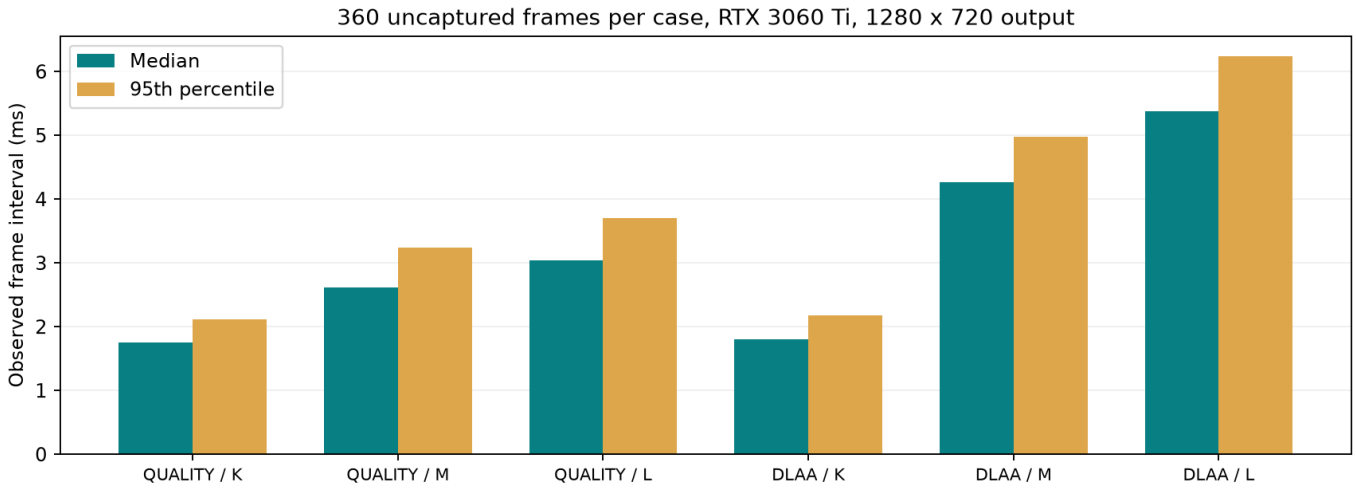
Practical recommendation

Keep Auto as the normal option and retain explicit models for comparison. Label model telemetry as requested unless independently confirmed. Include DLL version and driver in every visual issue report. M/L support on the RTX 3060 Ti does not mean their GPU cost is small.

Evidence / references: C15 DlssQualityPreset / DlssModelPreset / DlssNativeBridge; native options:274-293; [S4], [S9]; Streamline log in the evidence folder.

SHORT SAMPLES, NOT A PERFORMANCE CERTIFICATION

18. Observed cost and owned memory



Frame interval samples exclude screenshot capture. Each case used 240 warmup frames and 360 measured frames, in one fixed scene. Values are CPU-observed full-frame intervals, not isolated DLSS GPU timings.

Case	Median ms	p95 ms
QUALITY / K	1.753	2.114
QUALITY / M	2.617	3.234
QUALITY / L	3.045	3.699
DLAA / K	1.807	2.180
DLAA / M	4.264	4.977
DLAA / L	5.380	6.242

M and L increased observed frame intervals in this scene, especially in DLAA. These are brief samples on one GPU with normal clock and workload variation. They cannot be converted into a promised FPS gain, compared directly with an AAA title, or used to attribute every millisecond to DLSS.

Directly owned world targets were 10.54 MiB in 720p Quality and 19.34 MiB in 720p DLAA. The active 4x hand targets added 28.13 MiB in the hand test. These estimates exclude NGX allocations, driver pools, the main game target and other engine resources. The UI's AA allocation total is not total DLSS VRAM usage.

Evidence / references: timings.json; audit-native-hand-valid.json; C3 target formats; VulkanOwnedTargetMemory. No long-duration, multi-resolution GPU benchmark was attempted.

WHAT TO CHANGE FIRST

19. Prioritized findings and solutions

ID / priority	Finding and confidence	Recommended action
F1 / P1	Particle color has no matching object velocity. Measured in flame ROI.	Add particle motion with a depth/alpha policy, or composite selected effects after DLSS.
F2 / P1	Legacy bias-current-color mask is incompatible with the documented modern-model contract.	Remove it as a K/M/L mitigation; retire misleading controls and unnecessary history/mask work.
F3 / P2	Geometry matching is conditional; uncaptured renderer families, new topology and cap overflow fall back to camera flow. Code-confirmed.	Add coverage telemetry, renderer-specific support and a cap/LOD stress suite.
F4 / P2	Depth resource lacks explicit subresource metadata; SDK Vulkan fallback uses color aspect. Code-confirmed risk.	Chain correct aspect/range data and validate the native call under Vulkan validation layers.
F5 / P2	SDR-only color contract is not a general linear/HDR/shader-pack implementation.	Document support; add format/encoding/exposure integration per upstream renderer.
F6 / P2	MIP adjustment targets terrain only; no broad high-frequency texture validation.	Use native references and moving texture tests before extending bias or changing defaults.
F7 / P3	cameraPinholeOffset remains INVALID_FLOAT; SDK warning observed.	Set {0,0} for a centered camera, or supply a real offset if used.
F8 / P3	Requested model and partial memory totals can be mistaken for active model / total VRAM.	Keep telemetry labels precise and expose runtime/version and SDK memory separately.
F9 / P2 validation	Important lifecycle and camera-effect cases remain untested.	Add dimension/reload/camera-mod/minimize/device-loss coverage before calling the integration complete.

P1 means the current design cannot reasonably be treated as complete until resolved. P2 means a significant compatibility or quality gap, or a concrete risk requiring validation. P3 means a smaller correctness/diagnostic cleanup. Priority is not a claim that every item currently causes a visible artifact.

CONCRETE TESTS FOR THE PROPOSED FIXES

20. Acceptance plan for a follow-up patch

Area	Test / acceptance criterion
Camera and matrices	Retain the current yaw/pitch/strafe reprojection test. Expect errors near RG16F precision; investigate any coherent sign, scale or jitter-correlated pattern.
Particles	Use deterministic seeded replay. Compare raw color, depth and motion for a visible sprite; a moving particle must not inherit a stationary backdrop vector if it remains before DLSS.
Animated geometry	Exercise player swing/walk, animals, chest lid, dropped items, moving blocks, ropes, nameplates and modded renderer families. Log per-family match/skip counts.
Layered alpha	Overlap independently moving sprites and glass against thin geometry. Check alpha holes, depth edges, additive trails and particles behind walls.
Legacy mask removal	Compare modern models with a minimal documented input set. Confirm old sliders are removed or relabeled and that missing motion is fixed at its source.
Native metadata	Zero relevant validation-layer errors through creation, evaluation, resize, preset change and destruction. Depth range must be DEPTH with valid mip/layer bounds.
Color / exposure	Known grayscale and saturated ramps in SDR and any supported HDR path; test exposure changes, dark interiors and bright outdoor transitions.
Jitter / history	Full cycles: 8, 18/19, 24, 32 and 72 as appropriate. Test cuts, teleports, dimension changes, FOV effects and resource reload without periodic resets masking an input error.
Performance / memory	Measure isolated GPU passes and sustained full-frame performance with captures disabled. Include heavy entities and cap saturation, not only the simple fixture.

A useful quality threshold compares against a native-resolution reference under matched exposure, camera path and sharpening. Some reconstruction loss at low input resolution is expected. Do not define success as zero pixel differences, zero temporal variance, or a subjective claim of matching every AAA game.

WHAT THIS AUDIT DOES AND DOES NOT ESTABLISH

21. Reproduction, exclusions and confidence

The isolated client, per-scene settings, input logs and raw buffers remain under run/dlss-deep-audit-20261025/. The helper source, analysis and report generator are under build/dlss-deep-audit-20261025/. A compact, shareable evidence directory accompanies this PDF and includes figures, analysis, dispatch samples and the native log.

Accepted evidence	Meaning
static-*	32-frame stationary edge samples. Ultra Performance is a partial jitter-cycle sample.
camera-yaw / camera-pitch / camera-strafe	Controlled per-capture camera changes with raw depth and velocity readback.
entity-valid-*	Corrected direct-Java pig fixture; ON/OFF/ON velocity coverage check.
particle-k/m/l/dlaa and particle-off	Independent particle sequences. K includes raw velocity/mask; model images are illustrative, not synchronized quality scores.
chest-motion / third-person-swing	Animated geometry paths were exercised. Totals include other visible matched geometry.
underwater / native-hand-valid / texture-*	Visual composition and optional texture/sharpness coverage.
resize-* / dlaa-1080 / timing-*	Resolution changes and short uncaptured timing observations.

Unvalidated: Vulkan validation layers, deterministic cross-model replay, HDR shader packs, VR/multiview, multiplayer, full dimension/reconnect lifecycle, device loss, all camera effects, every custom renderer and vertex-cap saturation. The audit does not certify zero artifacts or parity with a named AAA game.

All 252 pre-existing production source/native files hashed before the audit were unchanged afterward. The normal run/26.3 config and options files also matched their fresh pre-audit hashes. The audit client stopped normally. No NVIDIA runtime DLL is included in the evidence deliverables.

```
Audited JAR SHA-256:
e1031416ded18d9fa45c8e77e07263633f48e1ba8824935c4f94d04e4249a152
```

```
Mod bridge SHA-256:
a4e7e9b3607a91941962b08debd5af0d3fc67606eb5037bea2b2cd045c998833
```

Evidence / references: validation.json contains full hashes, session metadata, exclusions and numerical results. The prior 133-test build belongs to the implementation baseline; this audit compiled only its disposable helper.

OFFICIAL DOCUMENTATION AND PINNED IMPLEMENTATION

22. Primary research sources

[S1] NVIDIA DLSS Programming Guide

31 March 2026, 84 PDF pages. Relevant: 3.1, 3.5-3.15, debugging guidance. The downloaded current copy exactly matches the pinned SDK PDF by SHA-256.

[S2] Streamline 2.14.1 DLSS integration

Resource tags, exposure path, constants, options, evaluation and lifecycle.

[S3] Streamline 2.14.1 common constants

Matrix layout, motion scaling and convention flags, camera metadata and defaults.

[S4] Streamline 2.14.1 DLSS options

Modes, K/M/L preset IDs, exposure/HDR options and DLSSState contents.

[S5] Streamline DLSS implementation

Preset recreation, resource conversion, NGX evaluation inputs and Vulkan fallback subresource range.

[S6] Streamline manual integration

Manual presentation bookkeeping and command-buffer state ownership.

[S7] Khronos VkImageSubresourceRange

Correct aspects and mip/layer bounds for Vulkan image subresources.

[S8] Streamline common validation

validateCommonConstants checks cameraPinholeOffset despite its optional header annotation.

[S9] NVIDIA model guidance

K/M/L intended scaling modes, RTX 20/30 performance cost and NVIDIA App override behavior.

Research checked 25 October 2026. This report uses production runtime captures, not a debug DLL watermark or a synthesized rendering. Technical requirements are paraphrased; the findings and measurements describe the audited mod.

SOURCE LOCATIONS FOR IMPLEMENTATION WORK

23. Repository reading map

Locations refer to the current working tree, whose content was hashed for this audit. Most classes below are under src/client/java/org/betterLostItems/salts_anti_aliasing/. The shader paths are under src/main/resources/assets/salts_anti_aliasing/.

C1 - mixin/client/GameRendererMixin.java

Scene entry 107-129; projection jitter 131-145; resolve after world 147-164; foreground hooks 56-80.

C2 - client/render/common/RenderRuntime.java

beginSceneRendering 494; endSceneRendering 516; native lifetime 483; foreground handling around 433.

C3 - client/render/vulkan/VulkanSceneDlssController.java

Scene configuration 77-114; JNI evaluate 215-309; barriers 323-343; mask 362-369; target formats 371-414.

C4 - client/render/vulkan/VulkanSceneTemporalController.java

Jitter 125-174; projection 180-193; camera arrays 486-553; frame capture 561-625; reset thresholds 40-42.

C5 - client/render/vulkan/TemporalCameraMatrices.java + shaders/include/temporal_motion.glsl

Double-precision inter-frame rebasing; dense unjittered current-to-previous screen motion.

C6 - client/render/vulkan/EntityMotionGeometry.java

Submission capture 74-91, 121-140; cap 100; vertex matching 193-216.

C7 - client/render/vulkan/VulkanEntityMotionRenderer.java

Matched geometry pass 50-185; two matching traversals; current/previous positions; sampled-depth rejection.

C8 - shaders/post/entity_motion.vsh and entity_motion.fsh

Current/previous clip motion, jittered rasterization and depth-validated fragment replacement.

C9 - client/render/vulkan/VulkanTemporalColorRejectionRenderer.java + shaders/include/temporal_color_rejection.glsl

Heuristic 3x3 color envelope, previous color, motion/jitter reprojection and mask controls.

C10 - mixin/client/ModelFeatureRendererMixin.java, ItemFeatureRendererMixin.java, MovingBlockFeatureRendererMixin.java, BlockEntityRenderDispatcherMixin.java

Specific captured geometry families; compare with the full client mixin list for absent paths.

C11 - client/config/NumericTuningSetting.java

DLSS Advanced rejection sliders at 83-86. These currently describe the legacy mask rather than a modern supported feature.

C12 - client/render/vulkan/VulkanHandAaController.java

Native-resolution foreground MSAA and independent target allocation.

C13 - client/render/vulkan/NativeFeatureRetirement.java

Inactive-frame and GPU-fence retirement.

C14 - client/render/vulkan/VulkanTextureDetail.java

Terrain-only pipeline selection and optional quarter-step MIP bias.

C15 - client/config/DlssQualityPreset.java; DlssModelPreset.java; client/render/vulkan/dlss/DlssNativeBridge.java

Stable native IDs, model-aware settings cache, DLAA dimension validation, ABI 3 and native runtime loading.

Native bridge: native/dlss_bridge/src/salts_dlss_bridge.cpp. Resource helper 84-103; options 258-315; evaluation 319-478; resource tags 419-425; constants 430-458. Pinpoint line numbers are advisory if the working tree is edited after this report.